National University of Singapore

Institute of Systems Science
Intelligent Reasoning Systems
Project report

GROUP PROJECT REPORT

AnchorAI

Explainable, budget-aware retention planning for gyms

Churn prediction · uplift estimation · rule-based reasoning · case-based reasoning
genetic algorithm with an exact CP-SAT benchmark · knowledge graph · grounded staff assistant

TEAM

Abdulaziz Ali S Hasanain	A0350856X
Asiri Mohammed Yahya A	A0351967N
Binmaneea Khalid Waleed M	A0355712E

National University of Singapore

Group 16 · 25 October 2026

Abstract

About one gym member in three leaves each year, and most stop attending weeks before they cancel. Retention tools flag members at risk but leave the decision that matters (whom to contact this week, with which offer, within a fixed budget and limited staff hours) to guesswork. AnchorAI makes that decision and explains it. A calibrated ensemble predicts churn on real members; a forward-chaining rule base encodes the gym's playbooks and guardrails; case-based reasoning estimates what each offer would achieve for each member; a genetic algorithm, benchmarked against an exact CP-SAT solver, builds the weekly plan; a knowledge graph adds signals the churn model cannot see; and a grounded assistant answers staff questions from the system's own evidence. Segmentation and churn are evaluated on 4,000 real members of a public dataset; decisions on a documented synthetic scenario layer whose true offer effects are known, which shows that the method works when effects are of this kind, not how large they are in a real gym.

0.908
churn F1 on 800 unseen real members (PR-AUC 0.973)

+22–138%
more members kept than contacting by risk or by schedule, at equal budget

0.24%
largest gap of the genetic algorithm to the proven optimum

96.2%
of the assistant's judged sentences supported by its evidence

Contents

1 Business case, market research and objectives . . . 1
2 System design . . . 3
3 Data and the scenario layer . . . 3
4 Methods . . . 5
5 System development and implementation . . . 7
6 Experiments and findings . . . 10
7 Findings, discussion and future work . . . 17
References . . . 17
A Project proposal . . . 19
B Mapping to MR, RS and CGS . . . 22
C Installation and user guide . . . 24
D Use of AI tools . . . 26
E Individual project reports . . . 29

1 Business case, market research and objectives

Gym members rarely cancel on impulse: they stop coming first and cancel weeks later, and that lag is the window in which a well-chosen contact can still keep them. The median member retention rate was 66.4% in 2024 across 175 operators [1], and new members are the most fragile: in one fitness centre 63% stopped before their third month [2]. No real operator was involved: the interviews that shaped the rules were simulated.

The proposal identified three ways in which current practice fails (Table 1). The common thread is that each produces either a list or a score, but not a decision: none says whom to contact this week, with which offer, within a fixed budget and finite staff hours, and why.

Table 1 Current retention practice and its failure modes, as identified in the proposal.

Practice	Why it fails
Calendar campaigns	Contact everyone on a schedule, so budget goes to members who would have stayed anyway.
Threshold rules	Ignore trend: twelve visits a month falling to four is a stronger warning than a steady two, but a threshold treats them alike.
Vendor risk scores	Give a number, not a decision; staff cannot interrogate the flag, so they ignore it.

1.1 Market and positioning

Memberships grew 6% and facilities nearly 4% in 2024 [3], so there are ever more members to keep. Club software divides into systems of record (membership, booking, payments, access control; for example ABC Fitness, Mindbody, Xplor, Perfect Gym, Virtuagym and Daxko) and an intelligence layer that bolts onto them, of which Keepme, with its predictive member score, is the clearest example; Glofox offers a built-in at-risk report. The proposal's desk research, based on vendor material and buyer guides that have a commercial interest, found four gaps:

1. **Scores, not allocation.** No vendor publicly exposes constrained allocation: which flagged members to contact, with which offer, under a budget and staff hours.
2. **Risk, not response.** Vendors target the members most likely to leave, but on field data, targeting by the expected response to the intervention reduces churn more than targeting by risk [4].
3. **Opaque scores.** Models trained on very large member pools are powerful but unauditable, and mid-market operators cite staff distrust of black-box flags.
4. **Priced for chains.** The intelligence layer is generally recommended from five locations upwards, which leaves the one-to-five-club operator underserved.

AnchorAI is therefore positioned as an *explainable, budget-aware retention planner for mid-market operators*. It does not try to out-predict incumbents with far larger training data; it differentiates on the decision and on its transparency. We have not tested any competitor product, so every comparison in this report is against baseline policies in our own experiments, not against a vendor.

1.2 Pricing and business value

AnchorAI is sold as an annual subscription that bolts onto the operator's existing membership system: US$199 a month for the first club and US$149 for each further club up to five, so a three-club operator, the size of our scenario, pays US$497 a month. It runs on the operator's own machine, language model included: in E8 a local seven-billion-parameter model was as faithful to its evidence as the hosted one (95.1% of judged sentences supported for each), so there is no fee per question. A hosted and maintained option would cost more, to cover hosting and support; we have not costed it.

The price sits where the gap in the market is (Table 2). The dedicated retention layer is priced for chains: in 2020 Keepme's founder quoted US$9,000 a year for a single site, with a lower price for each further site [5], about US$750 a month, nearly four times AnchorAI's first club. Small operators already pay at AnchorAI's level for their membership system and its add-ons [6, 7].

The value comes from members kept. The planner values a kept member at six months of fees, a deliberately cautious horizon; at the scenario's middle fee of US$54, for a six-month contract, that is US$324, an assumption of the scenario layer rather than a market figure. One club therefore breaks even when AnchorAI keeps 0.6 members a month who would otherwise have left, and three clubs need 1.8. In the scenario, at the default 36 staff hours and US$1,500, a week's plan from the greedy planner

Table 2 AnchorAI's price against published prices of club software, in US dollars a month.

Product	What it is	Price a month
AnchorAI	retention planner: first club; each further club, up to five	199; 149
Keepme	retention intelligence layer, single site (2020 figure) [5]	about 750
PushPress	membership system; marketing add-on [6]	159–229; 329
Gymdesk	membership system by member count; each extra location [7]	75–240; 50–100

on the case-based estimates keeps 21.0 members against 13.7 for contacting by risk, and spends US$432 instead of US$1,280 (E4): 7.3 more members from a single week, four times the monthly break-even for three clubs. The scenario's effects are synthetic, so this shows that the margin is wide, not how wide it would be in a real gym. The direction agrees with field evidence that targeting by expected response reduces churn more than targeting by risk [4], and the stake is large, since about a third of members leave each year [1].

Table 3 Objectives, how each is tested, and the verdicts argued in Section 6.

	Objective	Test and baseline	Verdict
O1	know who is at risk, and why	E1 segments (every k); E2 churn (visit rule, logistic regression); E7 graph signals	● Met
O2	know whom a contact would keep	E3 uplift (T-learner, risk, random)	● Partly met
O3	plan the week within budget, staff and stock	E4 plan value (contact-all, risk-top-k); E5 optimality and speed (CP-SAT)	● Met
O4	keep people in control	E6 rules and guardrails; tests of explanation and audit coverage	● Met
O5	usable by staff in plain language	E8 assistant (keyword router)	● Partly met
O6	reproducible and dependable	fresh rebuilds; tests with services down	● Met

2 System design

One web application serves administrators, managers and staff; one FastAPI service checks every request's role, audits every change and runs long jobs in the background; and three reasoning components do the work (Figure 1). They meet at three contracts: *data tables* written once by the data build and read by everything else; an *allowed-offer list* from the rule engine, so the optimiser never sees an offer the rules block; and an *explanation record* per plan item (drivers, rule trace, graph signals, ten similar past cases), which the plan's 'why' panel and the assistant both read, so they always give the same reason. Each week runs from scores to an approved, assigned call list, and logged outcomes feed the next week (Figure 2).

3 Data and the scenario layer

Segmentation and churn prediction use only the 4,000 real members of the public *Gym Customers Features and Churn* dataset [8]: 14 columns, 26.5% churners, split 3,200/800 with stratification, and four derived features such as the visit trend (this month's visits over the lifetime rate), whose median is 0.69 for members who left and 1.00 for those who stayed (Figure 3). Two same-month columns may leak the label, so E2 reports the model with and without them. The snapshot has no clubs, classes, offers or

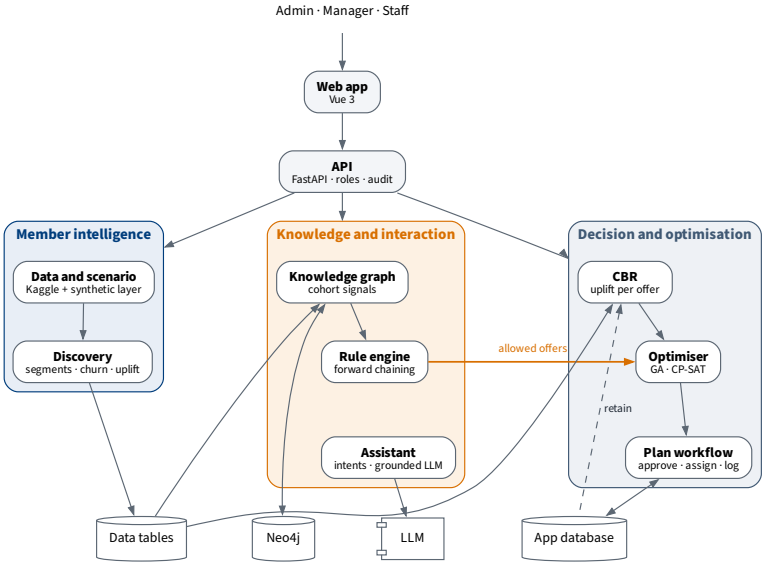


Figure 1 Architecture. Arrows show data flow; the dashed arrow is the learning loop through which logged outcomes return to the case base.

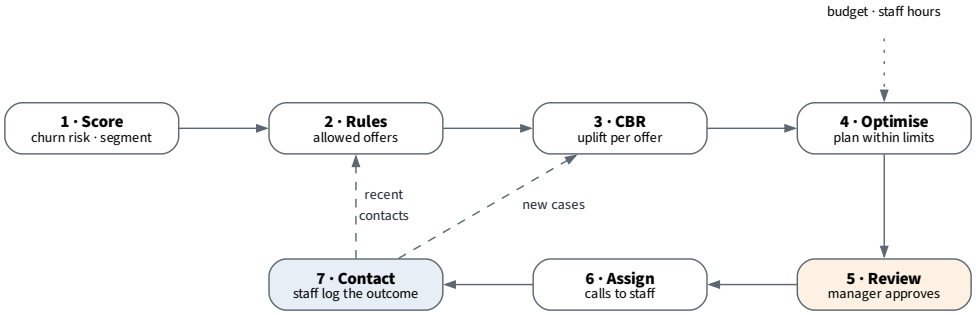


Figure 2 The weekly cycle: steps 1–4 run automatically within the week's budget and staff hours; the manager reviews and approves, and staff contact members and log outcomes.

contact history, so a documented *scenario layer* attaches a synthetic gym: three clubs, nine instructors and 18 classes consistent with each member's real visits; four offers with costs, staff minutes and stock; a heterogeneous ground-truth uplift for all 16,000 member–offer pairs (mean +5.8 points); two gym events (an instructor leaving and a class cancelled); and a randomised history of 2,400 past contacts (Figure 4). The ground truth is used only to grade, never by an estimator or planner. One command rebuilds every table, model and score in about 20 seconds from one seed.

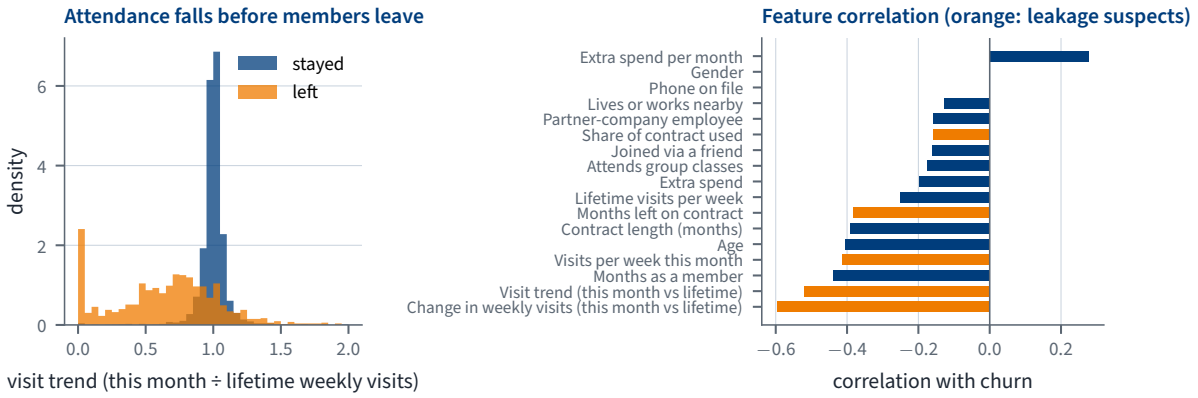


Figure 3 Left: the visit trend of members who stayed clusters at 1, while members who left spread below it, with a spike at 0 for those who did not come this month (medians 1.00 and 0.69). Right: correlation of each feature with churn; the two strongest are leakage suspects. Real Kaggle data.

Source: `scripts/figures.py` (trend), as in `a_features_leakage.ipynb`, cells 2–4.

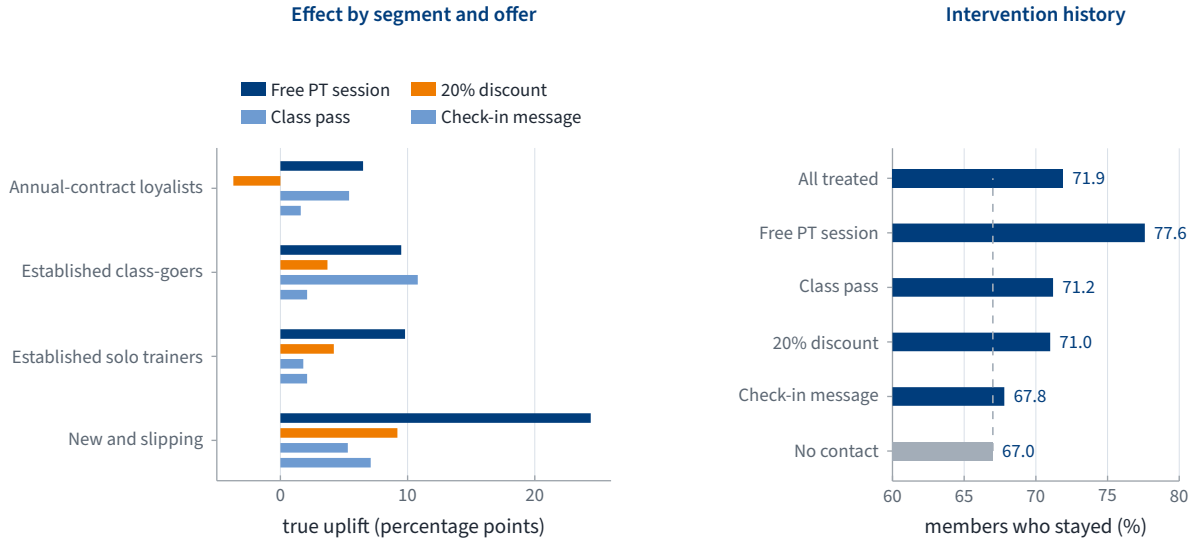


Figure 4 The scenario's ground truth. Left: mean true uplift by segment and offer; a discount backfires on annual members. Right: share who stayed in each arm of the randomised history against the 984 controls (dashed).

Source: `results/a5_true_uplift_by_segment.csv`; `a_scenario_uplift.ipynb`.

4 Methods

4.1 Member intelligence

K-means [9] on seven behavioural features gives segments that staff can name; k is chosen by a fixed rule on the silhouette [10] and names come from rules on the centres, so they survive a rebuild. Churn is predicted by the mean of a class-weighted logistic regression and LightGBM [11] over 17 features (Figure 5). Class weights matched SMOTE-NC [12] in cross-validation and were kept as simpler; a Platt calibrator [13] fitted on a held-out slice makes the probabilities honest, which matters because the planner multiplies them by uplift and value. Training members are scored out of fold, and each member's five drivers average the logistic-regression term with LightGBM's TreeSHAP value [14] (Figure 6). A T-learner [15], one outcome model per offer and one for controls, is the uplift benchmark.

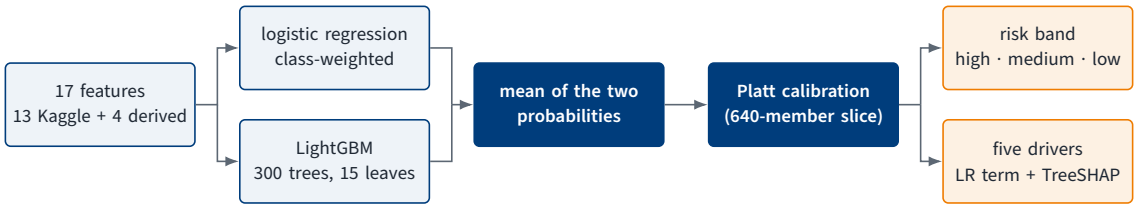


Figure 5 The churn pipeline, fitted on 2,560 training members and calibrated on the remaining 640.

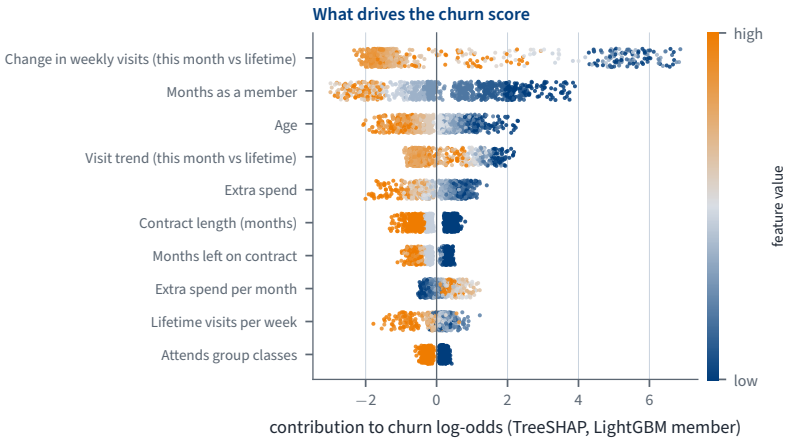


Figure 6 TreeSHAP values of the LightGBM member on the 800 test members, for the ten most influential features. Each dot is a member, coloured by its feature value; dots to the right push towards churn. Real Kaggle data.
Source: `scripts/figures.py` (churn), as in `a_churn.ipynb`, cell 5.

4.2 Decision automation

A forward-chaining engine [16] runs 29 YAML rules: 8 *derive* rules name patterns (falling regular, new member, routine disrupted), 16 *playbooks* propose an offer on a channel, with twins for members without a phone, and 5 *guardrails* block offers (contact and offer caps, eligibility, no paid offers at low risk, no repeat training session). Conflicts resolve by salience, specificity and order; each rule fires once, and blocks are applied after the loop, so guardrails always win. Every rule cites its source, a validator rejects inconsistent rule bases, and administrators edit, test, version and roll back rules in the application. Thresholds were drafted on the training data and adjusted after three *simulated* manager interviews, which are disclosed as plausible opinion, not evidence.

4.3 Decision and optimisation

Each allowed member–offer–channel option is valued as $v_{mo} = \hat{u}_{mo} f_m H - c_o$, the estimated uplift [17] times the monthly fee over a cautious six-month horizon, minus the offer's cost. The uplift comes from case-based reasoning [18] (Figure 7): retrieve the 200 nearest past cases in one shared neighbourhood of treated and untreated members, reuse it as a local A/B test, revise by empirical-Bayes shrinkage towards the offer's average [19], and retain logged outcomes as evidence but never in the averages, because the planner's own choices would bias them. A null test with shuffled offers rejected an earlier design that measured risk instead of effect. The plan maximises total value within budget, staff minutes, trainer minutes and stock, plus two effects between members who train together: *spillover* and *diminishing returns* per instructor. These make the problem non-linear, so a genetic algorithm (DEAP [20]) searches it with a repair operator and a greedy seed, while CP-SAT [21] solves the linear case exactly and

benchmarks the full one on small instances. A policy evaluator grades plans against the true effects and is never used to choose one.

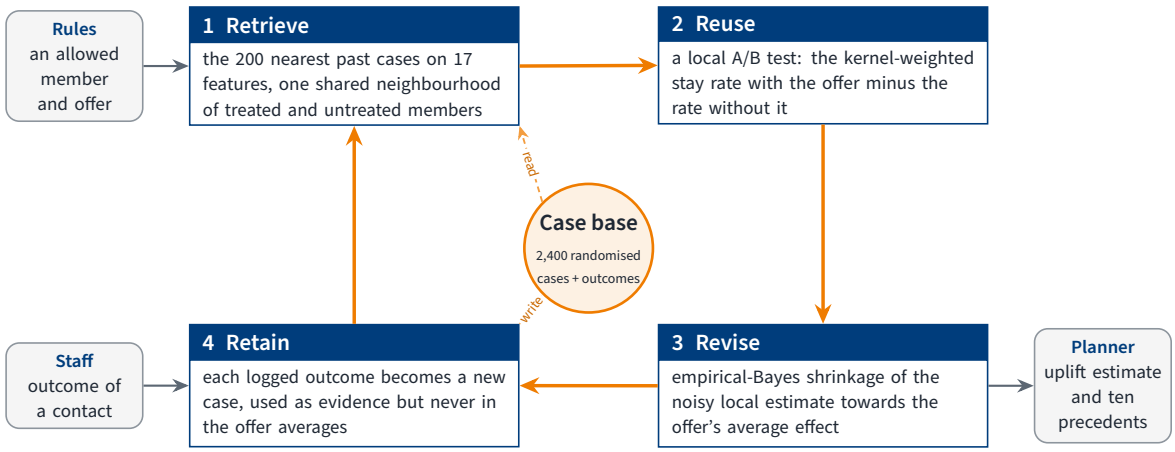


Figure 7 The case-based reasoner. The neighbours that produce an estimate are shown to staff as its precedents.

4.4 Knowledge graph and staff assistant

A Neo4j knowledge graph [22] links members, classes, instructors, clubs, offers and events through read-only parameterised Cypher. It flags members whose anchor instructor left or whose class was cancelled, as rule facts cached so the rules work without Neo4j; it gives the optimiser its class cohorts; and it simulates an event's impact without changing data (Figure 9a). The staff assistant routes a question to one of seven intents with sentence-embedding centroids [23] and an 'unknown' threshold, fills slots, runs one deterministic tool, and lets the language model only phrase the numbered evidence with citations, or answer that it does not know (Figures 8 and 9b). Without the model, it returns the cited evidence itself.

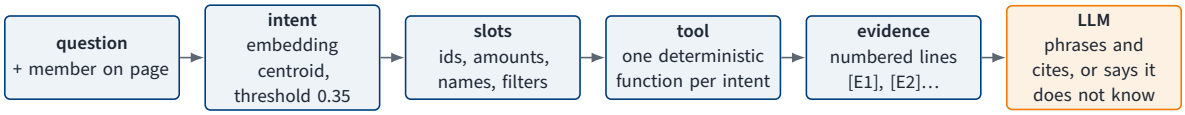
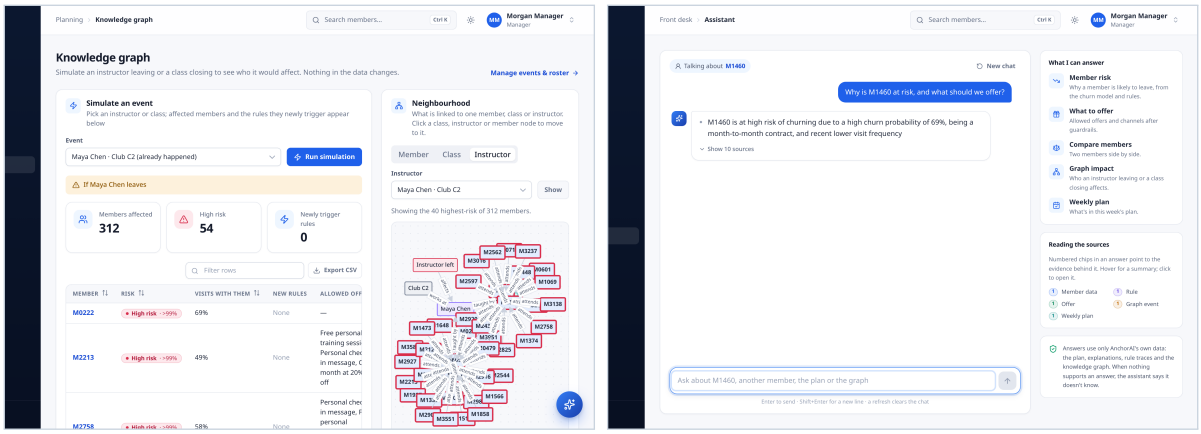


Figure 8 The assistant pipeline: the language model never chooses a tool and never sees data a tool did not return.

5 System development and implementation

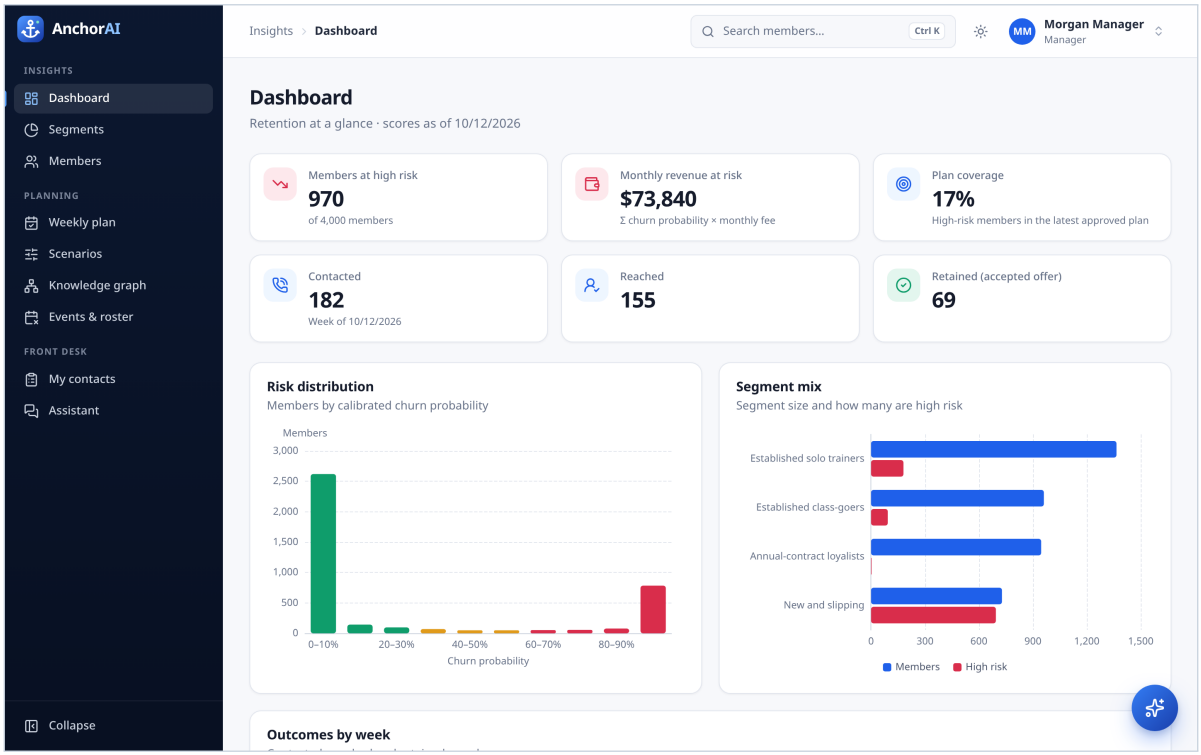
One command builds the data and models, migrates and seeds the database with a demo week, and loads the graph. Sixteen pages serve three roles (Figure 10): managers work from the dashboard, plan and scenarios; staff from a call sheet with talking points, message drafts and two-tap outcome logging; administrators from versioned rules, offers, data and models, users and an audit log of every write. Member 360 shows every component's reasoning about one member on one screen. Risk is always shown as text and colour; planning continues when Neo4j or the language model is down; and 195 offline tests guard against leakage, infeasible plans, rule order, unsafe queries and role or audit errors.

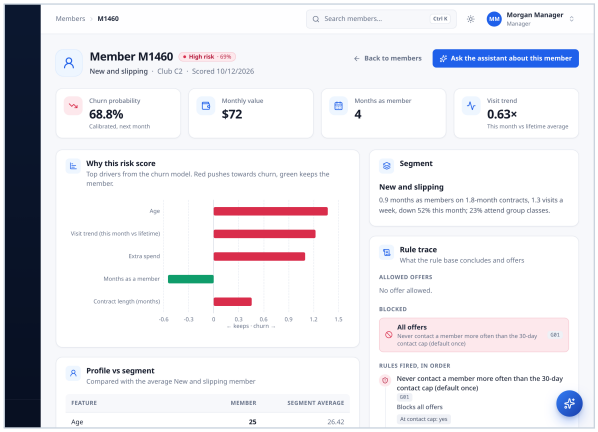


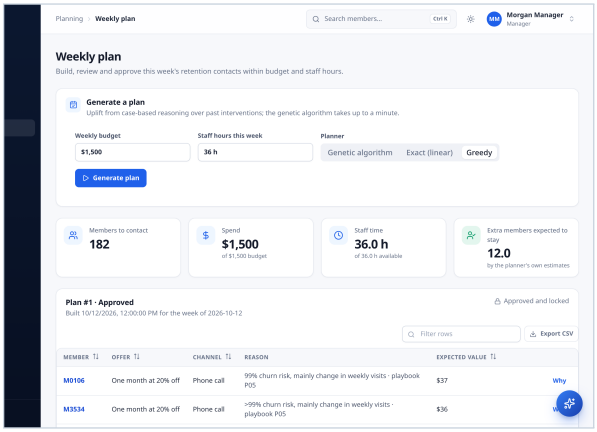
(a) Simulating instructor Maya Chen leaving: 312 members affected, 54 at high risk, with her neighbourhood.

(b) The assistant answering about member M1460 with the local `qwen2.5:7b-instruct` model, citing its sources.

Figure 9 The knowledge graph and the staff assistant in the application.







(a) Member 360: risk, drivers, profile and rule trace for one member. **(b)** The weekly plan: 182 approved contacts, each with a reason.

Figure 10 The application on the demo data. Top: the manager's dashboard.

6 Experiments and findings

Eight experiments test the objectives. E1 and E2 use only the 4,000 real members. E3–E7 are graded against the scenario layer's ground truth, which no estimator or planner ever reads, so they show that the system recovers effects of this kind, not how large real effects are. E8 evaluates the assistant on the system's own evidence (Table 4). Every number below comes from a committed results file or notebook, all reproducible from seed 42.

Table 4 The eight experiments: real-data experiments use the 4,000 Kaggle members; scenario experiments are graded against the synthetic ground truth.

	Question	Key result	Verdict
E1	meaningful segments? REAL DATA	$k = 4$ by rule; stability ARI 1.00; churn from 94.6% to 1.4%	● Met
E2	churn model beats simple rules? REAL DATA	F1 0.908, PR-AUC 0.973, Brier 0.035 vs 0.862 / 0.953 / 0.056 (logistic) and 0.549 / 0.390 / 0.336 (visit rule)	● Met
E3	who is persuadable? SCENARIO LAYER	CBR ties the T-learner (Spearman 0.39 each), better for PT and discount	● Partly met
E4	more members per dollar? SCENARIO LAYER	22–138% more members kept than by risk or schedule; 0 of 198 plans infeasible	● Met
E5	GA good and fast enough? SCENARIO LAYER	within 0.24% of the optimum on small instances; 33 s for 2,000 members	● Partly met
E6	rules correct; guardrails matter? SCENARIO LAYER	0 validator issues; 29 of 29 cases; guardrails remove 31.3% of proposals	● Met
E7	graph signals add information? SCENARIO LAYER	no ranking gain; 243 extra low-risk members reached; +7 members at 120 h	● Partly met
E8	assistant accurate, grounded? SCENARIO LAYER	96.2% of sentences supported; held-out routing 50.0% vs 54.2% for keywords	● Partly met

Source: `SystemCode/notebooks/results/`; method, baseline and caveats for each experiment follow.

6.1 E1: Are the segments meaningful?

REAL DATA Baseline: every k from 2 to 10

K-means was fitted on the 3,200 training members for each k and scored by silhouette, by stability across five other seeds (adjusted Rand index) and by how sharply the segments separate churn, which the clustering never sees.

Results. The documented rule chooses $k = 4$ (silhouette 0.174 against the best of 0.178 at $k = 3$); $k = 4$ adds a distinct annual-contract segment. Stability is very high: the mean adjusted Rand index across seeds is 1.00 (minimum 0.998). Test members fall into the segments in proportions within 4.2 percentage points of the training split, and churn separates by a factor of almost 70 (Figure 11). The 728 'new and slipping' members hold about two-thirds of all churners.

Verdict ● Met. The segments are not geometrically crisp, as expected for behavioural data, but they are chosen by rule, stable, carry over to unseen members and separate risk in terms staff can name.

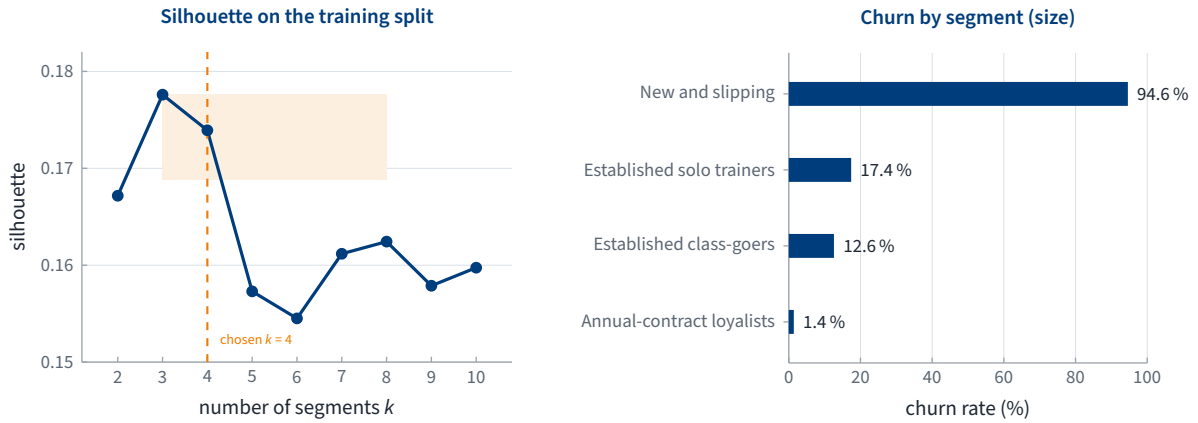


Figure 11 E1. Left: silhouette is low and nearly flat; the shaded band is within 5% of the best k between 3 and 8, and the rule picks the most detailed k inside it. Right: the four segments separate churn from 94.6% to 1.4% (sizes 728, 1,365, 961 and 946).
Source: `results/e1_silhouette_by_k.csv`, `results/e1_segment_profiles.csv`; `a_segmentation.ipynb`.

6.2 E2: Does the churn model beat simple rules?

REAL DATA Baselines: a visit-threshold rule; a single logistic regression

All metrics are on the 800 untouched test members, with churn as the positive class. A model that beats the baselines should do so on all three of F1 (the yes-or-no flag), PR-AUC (the ranking, suited to an imbalanced label [24]) and the Brier score [25] (the probabilities the planner consumes).

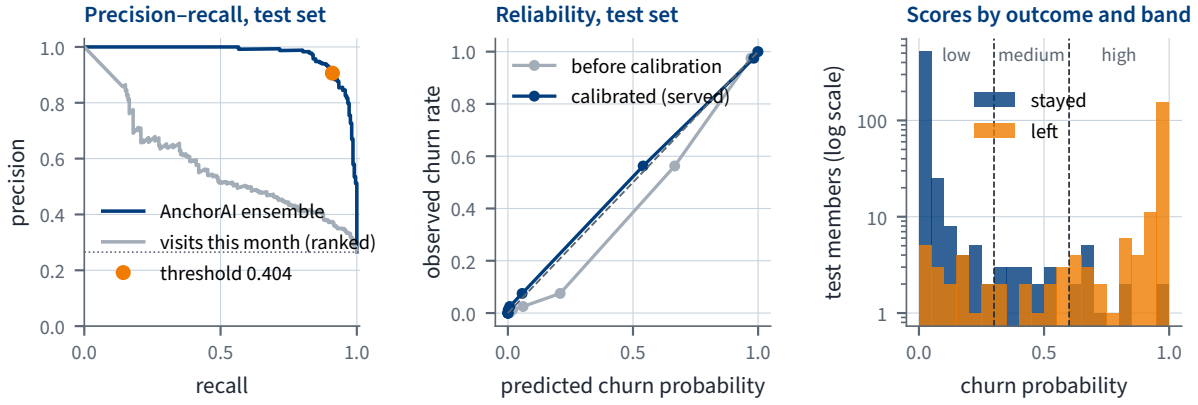


Figure 12 E2 on the test set. Left: precision–recall of the ensemble against members ranked by this month's visits, with the operating point at the threshold 0.404. Centre: reliability before and after Platt calibration. Right: scores by actual outcome, with the band cut-offs at 0.3 and 0.6.
Source: `scripts/figures.py` (churn), as in `a_churn.ipynb`, cells 3–4.

Table 5 E2. Test-set performance. The visit rule flags members with at most 1.63 visits a week this month, a threshold chosen on the training split.

Model	F1	Precision	Recall	PR-AUC	ROC-AUC	Brier
AnchorAI ensemble, calibrated	**0.908**	**0.906**	**0.910**	**0.973**	**0.988**	**0.035**
Single logistic regression	0.862	0.818	0.910	0.953	0.978	0.056
Visit rule	0.549	0.426	0.774	0.390	0.699	0.336
Ensemble without the five suspect features	0.817			0.900		0.069

Source: `results/e2_model_vs_baselines.csv`, `results/a_leakage.csv`.

Results. The ensemble beats both baselines on every metric (Table 5). At the threshold of 0.404 it flags 193 of the 212 test churners with 20 false alarms: about one flag in eleven is a member who stayed. Calibration moves the curve onto the diagonal (Figure 12), so each band churns at about its predicted rate: high-band members churn at 93.9% against 95.1% predicted, medium at 41.7% against 44.1%, low at 2.9% against 1.6%. Without the suspect same-month columns, F1 falls to 0.817, the honest lower bound if those columns were recorded after the churn month. Recall is 0.985, 0.943 and 0.522 for ages 18–25, 26–30 and 31+: the model misses about half of the rare churners aged 31 and over (23 in the test set); we report this rather than hide it, and suggest staff review medium-risk members of that group.

Verdict ● Met. F1 0.908, PR-AUC 0.973 and Brier 0.035 against 0.862, 0.953 and 0.056 for logistic regression and 0.549, 0.390 and 0.336 for the visit rule, with calibrated probabilities the planner can use at face value.

6.3 E3: Can we find who is persuadable?

SCENARIO LAYER Baselines: T-learner; churn risk used as uplift; random

Estimates for all 16,000 member–offer pairs were compared with the true uplift by rank correlation and by the mean true uplift of each method's top 10% of pairs; the same estimates then drove the greedy planner, graded by the evaluator.

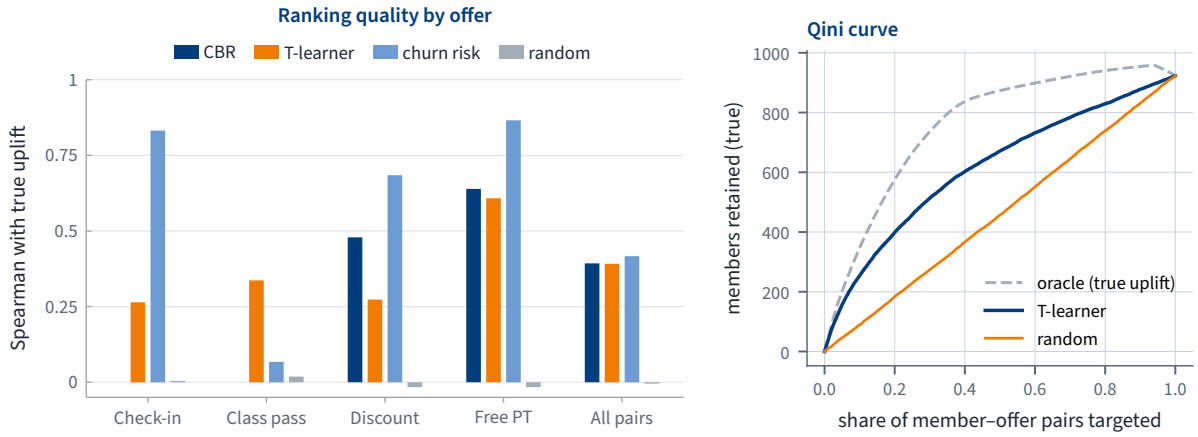


Figure 13 E3. Left: rank correlation between estimated and true uplift; CBR gives every member the offer's average for the check-in and the class pass, so it has no ranking there. Right: Qini curve of the T-learner against the oracle and random targeting.

Source: `results/e3_uplift_vs_truth.csv`, `results/a6_tlearner_recovery.csv`; `b_cbr.ipynb`, `a_scenario_uplift.ipynb`.

Results. CBR ties the T-learner overall (Spearman 0.39 each), is better for the two large-effect offers (free PT 0.64 against 0.61; discount 0.48 against 0.27) and picks the pairs with the highest true effect (top-decile mean 0.177 against 0.159). It cannot separate members for the check-in and the class pass, whose effects of a few points are smaller than the noise in about 350 randomised cases per offer, so shrinkage honestly returns the average. Churn risk ranks well *within* an offer, because scenario effects grow with risk, but says nothing about *which* offer helps, so its top picks are weaker (0.125). Plans built on the estimates (Table 6) favour the T-learner at the default 36 staff hours, where cheap check-ins dominate and CBR cannot rank them, and CBR with more staff time. A null test with shuffled offers confirms that CBR does not invent effects.

Verdict ● Partly met. CBR matches the T-learner overall, is better where effects are large and explains each estimate with real precedents; it is weaker where effects are small and staff time is scarce, which costs 3.7 retained members a week at the default settings. The team kept CBR as the planner's estimator for its inspectable precedents, with the T-learner as its benchmark.

6.4 E4: Does AnchorAI retain more members per dollar?

SCENARIO LAYER Baselines: contact-all; risk-top-k; greedy-uplift

Every planner ran at 36, 72 and 120 staff hours and several budgets, the genetic algorithm with three seeds, and every plan was graded by the evaluator as extra members kept beyond the roughly 2,660 of 4,000 who would stay anyway. Variant (a) gives every planner the true effects, isolating search quality; variant (b) gives them the estimates, as in real use. None of the 198 plans broke a limit.

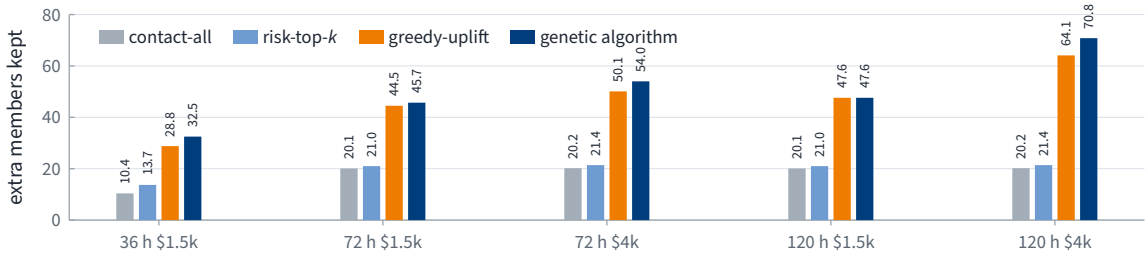


Figure 14 E4 (a), perfect knowledge: extra members kept by each planner, by staff hours and weekly budget. Planning by effect keeps 1.9 to 3.5 times as many members as contacting by schedule or by risk. The estimates behind the system's own plans are in Table 6.

Source: `results/e4_optimiser.csv`, mean over three seeds; `b_ga.ipynb`.

Table 6 E4 (b), estimated effects: extra members kept, as the system runs. Default configuration: the genetic algorithm on CBR estimates.

	Contact-	Risk-	CBR		T-learner	
Staff hours, budget	all	top-k	greedy	GA	greedy	GA
36 h, $1,500	10.4	13.7	21.0	16.7	24.6	**24.9**
72 h, $1,500	20.1	21.0	**36.6**	35.5	35.1	35.1
120 h, $4,000	20.2	21.4	**51.9**	48.1	50.7	50.7

Source: `results/e4_estimates.csv`, mean over seeds.

Results. With perfect knowledge the genetic algorithm beats greedy wherever two limits bind together and ties where one dominates (Figure 14). On estimates (Table 6), every effect-based plan beats contacting by schedule or risk, and per staff hour at the default week the default configuration keeps 0.46 members against 0.29 for contact-all and 0.38 for risk-top-k. But the genetic algorithm's advantage reverses on noisy estimates: against greedy, its true net value ranges from +2.2% to +9.2% with perfect knowledge, −1.5% to +0.4% on the T-learner and −33.8% to −1.0% on CBR. This is the optimiser's curse [26]: the harder a planner optimises noisy estimates, the more it selects the options whose estimates happen to be too high.

Verdict ● Met. Planning by estimated effect, as the system runs by default, keeps 22% to 138% more members than contacting by risk or schedule in every setting tested, without breaking a limit. The size of the gain depends on the estimates more than the optimiser: on today's CBR estimates, the simpler greedy planner, selectable on the same page, keeps more members than the default genetic algorithm.

6.5 E5: How good and how fast is the genetic algorithm?

SCENARIO LAYER Benchmark: CP-SAT's proven optimum on the planner's objective

The genetic algorithm was compared with CP-SAT on the planner's own objective: the linear objective on the full gym, and the full objective (with spillover and diminishing returns) on resampled instances small enough to prove optimality.

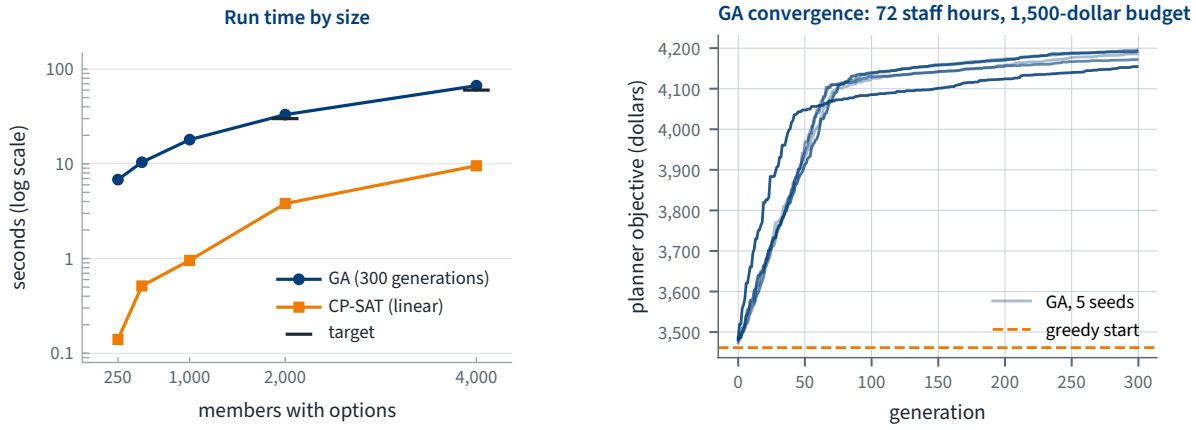


Figure 15 E5. Left: run time grows about linearly; this run missed the 30 s and 60 s targets by about 10%. Right: five seeds climb from the greedy start and plateau about 20% above it (regenerated on the seeded database; the committed run's five seeds end within 0.2% of each other).

Source: `results/e5_speed.csv`; `scripts/figures.py` (ga), as in `b_exact.ipynb`.

Results. On the linear objective, CP-SAT proves the optimum for the full gym in 1.2–1.4 s; the genetic algorithm is within 0.8–2.3% of it and greedy within 15–32%. On the full objective, the genetic algorithm matches the proven optimum on 8 of 9 instances of 20–40 members and comes within 0.24% at 50; at 200 members it beats CP-SAT's time-limited best by 1.0%, though at 100 it is 2.5% below. Greedy is 14–27% below the optimum on average. Its advantage over greedy holds, at 16–26%, for every combination of spillover $\beta \in \{0, 0.5, 1\}$ and diminishing returns $d \in \{0, 0.02, 0.05\}$. It takes 18 s for 1,000 members and 33 s for 2,000 (Figure 15).

Verdict ● Partly met. On the linear problem CP-SAT is the better tool, exact and faster. The genetic algorithm earns its place on the full objective at real size, where it is near-optimal and robust to the assumed effect strengths; its speed is at the edge of the targets.

6.6 E6: Are the rules correct, and do guardrails matter?

SCENARIO LAYER Baseline: the same playbooks with guardrails switched off

The full rule base ran on all 4,000 members with facts built exactly as the API builds them: the validator, the 29 hand-built cases, coverage, and 60 plans with and without guardrails, scored by the true uplift.

Results. The validator reports no issues, all 29 cases match, and every rule fires for at least one member. Guardrails remove 31.3% of proposals (Figure 16); every guardrail-free plan contacts members again within 30 days (75–148 times per plan at 36 h), and no guarded plan contains a violation. The 386 members without a phone receive the email or in-person twin of each playbook, never an SMS or a call. At 36 staff hours, where staff time binds, removing the guardrails changes plan value by −1.0 to



Figure 16 E6. Left: of 3,448 proposals over 1,422 members, guardrails remove 1,080 (31.3%), almost all of them contacts inside the 30-day cap. Right: their cost in plan value as the scenario measures it, which has no contact-fatigue effect and so cannot see what the cap protects against.

Source: `results/e6_guardrail_bites.csv`, `results/e6_plan_value.csv`; `c_e6_rules.ipynb`.

+1.4 members; at 120 h the guardrail-free plans gain 6–14 members by re-contacting members inside the cap.

Verdict ● Met. The rule base is consistent and its guardrails do real work. Their cost is zero to small at the seeded staff level and up to about 14 members at 120 h, but the scenario has no fatigue effect to price, so we claim safety and auditability, not a retention gain.

6.7 E7: Do knowledge-graph signals add information?

SCENARIO LAYER Baseline: churn probability alone

The graph's event flags were tested against the scenario's true post-event hazard (the Kaggle label knows nothing of the synthetic events): their lift, whether adding them to the churn ranking surfaces higher-hazard members among the 800 test members (1,000 bootstrap resamples), and their effect on rules and plans.

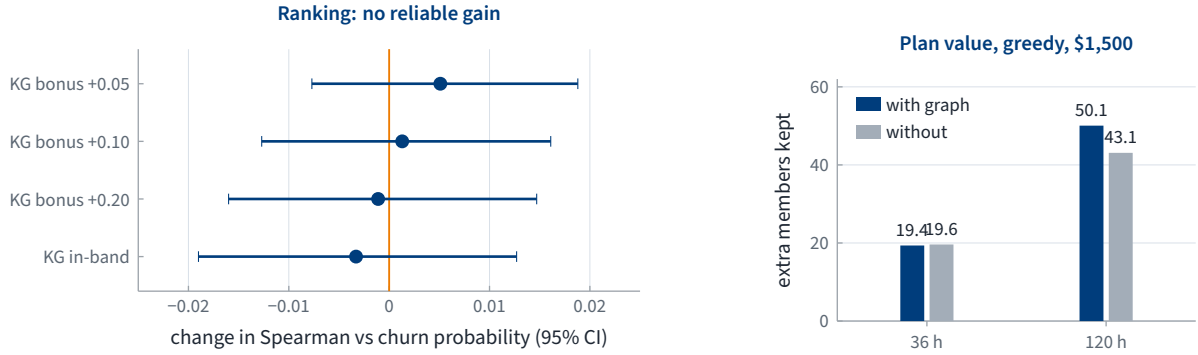


Figure 17 E7. Left: adding the graph's event flags to the churn ranking changes Spearman correlation with the true hazard by less than its uncertainty, whatever the bonus. Right: plan value of the greedy planner, the best in every setting, with and without the graph facts; the gain appears only when staff time is spare.

Source: `results/e7_ranking.csv`, `results/e7_plan_value.csv`; `c_e7_kg.ipynb`.

Results. The flags pick out the members the events hurt: their hazard rises 1.80 times after an anchor instructor leaves and 1.32 times after a class is cancelled, exactly the scenario's designed effects. They do not improve the ranking: every difference has an interval that includes zero (Figure 17), because

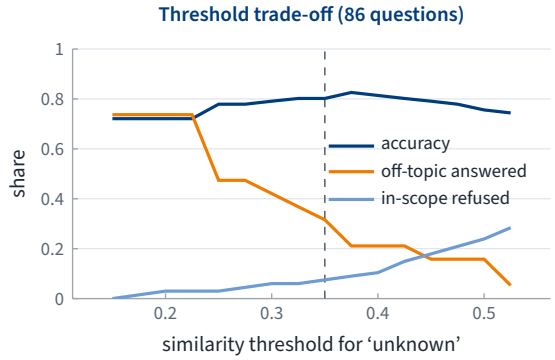
only about 8% of test members are flagged and the churn model already explains most of the hazard. They do change who is contacted: 243 low-risk members whose routine was just disrupted, a group the churn score misses, gain an allowed free check-in. At 120 staff hours contacting them adds about 7 members (+17%); at the seeded 36 hours the plan is already full and they add nothing.

Verdict ● **Partly met**. The graph carries information the model lacks when events of this kind happen, and turns it into contacts the churn score would miss. That becomes retention only when staff time allows, and the evidence is a recovery of a designed effect, not of real churn.

6.8 E8: Is the assistant accurate and grounded?

SCENARIO LAYER Baseline: a keyword router with the same slot extraction

Intent accuracy was measured on 86 labelled questions: 62 written by an AI model from the intent descriptions, and 24 held out, written by two team members who never saw the training examples. Twenty sampled questions ran through the full pipeline; each sentence of the 17 model-phrased answers was labelled supported, unsupported or contradicted by an LLM judge [27], after calibrating it on known good and bad answers, and 15 answers were spot-checked by hand.



Questions	n	Classifier	Keywords
AI-written	62	**91.9**	77.4
Held out	24	50.0	**54.2**

Intent accuracy (%).

Figure 18 E8: raising the threshold refuses more off-topic questions but also more in-scope ones; on held-out questions the classifier is no better than keywords.

Source: `results/e8_threshold_sweep.csv`, `results/e8_intent_predictions.csv`.

Results. Grounding is strong. No answer contained a number or member identifier absent from its evidence; 96.2% of the 80 judged sentences were supported and none contradicted; 15 of 17 answers were fully supported, and 13 of 15 were judged faithful by hand. A local `qwen2.5:7b` model was as well supported as the hosted model (95.1% each) and far faster on a strong GPU (median 0.23 s against 4.6 s), but passed fewer format checks (71% against 88%). Intent routing is the weak point (Figure 18): the classifier is near 92% on AI-written questions but 50% on held-out ones, against 54% for keywords, mostly because it answers off-topic questions instead of refusing.

Verdict ● **Partly met**. Answers are faithful to the evidence they are given, but the router that chooses that evidence is no better than keywords on unseen questions, so we do not claim the classifier is an improvement. Samples are small, the judge varies by about two points between runs, and the spot check was assisted by an AI tool, so it is not an independent human rating.

7 Findings, discussion and future work

Findings. AnchorAI shows that a gym's weekly retention decision can be automated and explained end to end. Real data supports a calibrated churn model and interpretable segments, with most churn in one recognisable group of new members whose visits are falling. Planning by estimated effect keeps substantially more members than planning by risk, but the quality of the uplift estimates, not the strength of the search, limits the planner. The rules and graph add safety and reach rather than accuracy, and the assistant is faithful to its evidence but routes unseen questions no better than keywords.

Limitations.

- **Synthetic ground truth.** Uplift, events and the contact history come from a documented scenario layer whose effects grow with risk by design. E3–E7 therefore show that the system recovers effects of this kind, not their real size; no real plan outcome exists.
- **One real snapshot.** The Kaggle data has no dates, so there is no out-of-time validation, recency or regularity feature, or survival model, and whether the strongest same-month columns leak cannot be settled from the data; both model versions are reported.
- **No real users.** The interviews that shaped the rules were simulated, and usability was assessed by design and by the assistant's evaluation, not with gym staff.
- **Small evaluation samples.** The assistant's held-out set has 24 questions and its faithfulness 17 judged answers; the medium risk band and the over-31 churners are small groups.
- **Untestable assumptions.** The scenario has no contact fatigue and no spillover, so it cannot reward the guardrail that prevents the one or the planning term that models the other.

Recommendations for an operator. Act on new members whose visits fall, since they are most of the churn and the driver chart shows why each is flagged. Review medium-risk members aged 31 and over by hand, where the model misses most. Keep visit dates and the outcome of every retention contact. Above all, hold back a small random control group each week, so that the real effect of each offer can be measured instead of assumed; until then, prefer the greedy planner, which the evidence favours on today's estimates.

Future work. Time-stamped data and randomised contacts to learn real effects; uncertainty-aware planning that stops a strong optimiser amplifying noisy estimates; out-of-scope detection for the assistant; drift monitoring after each rebuild; and a field study with gym staff.

References

[1] Health & Fitness Association. *2025 Fitness Industry Benchmarking Report*. 2025. URL: https://www.healthandfitness.org/2025-fitness-industry-benchmarking-report/.

[2] S. Sperandei, M. C. Vieira and A. C. Reis. 'Adherence to Physical Activity in an Unsupervised Setting: Explanatory Variables for High Attrition Rates among Fitness Center Members'. In: *Journal of Science and Medicine in Sport* 19.11 (2016), pp. 916–920. DOI: 10.1016/j.jsams.2015.12.522.

[3] Health & Fitness Association. *2025 Global Fitness Industry Report Shows Record Growth and What's Next for the Market*. Summary of the 2025 HFA Global Report. 18th Aug. 2025. URL: https://www.healthandfitness.org/2025-global-fitness-industry-report-shows-record-growth-and-whats-next-for-the-market/.

[4] E. Ascarza. 'Retention Futility: Targeting High-Risk Customers Might Be Ineffective'. In: *Journal of Marketing Research* 55.1 (2018), pp. 80–98. DOI: 10.1509/jmr.16.0163.

[5] Fit Tech. *Fit Tech People: Ian Mullane, Founder, Keepme*. Fit Tech, 2020 issue 2. 2020. URL: https://www.fittechglobal.com/fit-tech-features/Fit-Tech-People-Ian-Mullane-Founder-Keepme/34579.

[6] PushPress. *Pricing*. URL: https://www.pushpress.com/pricing (visited on 23/10/2026).

[7] Gymdesk. *Pricing*. URL: https://gymdesk.com/pricing (visited on 23/10/2026).

[8] A. Vinueza. *Gym Customers Features and Churn*. Kaggle dataset, CC BY-NC-SA 4.0. URL: https://www.kaggle.com/datasets/adrianvinueza/gym-customers-features-and-churn.

[9] D. Arthur and S. Vassilvitskii. 'k-means++: The Advantages of Careful Seeding'. In: *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*. 2007, pp. 1027–1035.

[10] P. J. Rousseeuw. 'Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis'. In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65. DOI: 10.1016/0377-0427(87)90125-7.

[11] G. Ke et al. 'LightGBM: A Highly Efficient Gradient Boosting Decision Tree'. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[12] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer. 'SMOTE: Synthetic Minority Over-sampling Technique'. In: *Journal of Artificial Intelligence Research* 16 (2002), pp. 321–357. DOI: 10.1613/jair.953.

[13] J. Platt. 'Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods'. In: *Advances in Large Margin Classifiers*. MIT Press, 1999, pp. 61–74.

[14] S. M. Lundberg et al. 'From Local Explanations to Global Understanding with Explainable AI for Trees'. In: *Nature Machine Intelligence* 2.1 (2020), pp. 56–67. DOI: 10.1038/s42256-019-0138-9.

[15] S. R. Künzel, J. S. Sekhon, P. J. Bickel and B. Yu. 'Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning'. In: *Proceedings of the National Academy of Sciences* 116.10 (2019), pp. 4156–4165. DOI: 10.1073/pnas.1804597116.

[16] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2021.

[17] N. J. Radcliffe. 'Using Control Groups to Target on Predicted Lift: Building and Assessing Uplift Models'. In: *Direct Marketing Analytics Journal* 1.3 (2007), pp. 14–21.

[18] A. Aamodt and E. Plaza. 'Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches'. In: *AI Communications* 7.1 (1994), pp. 39–59. DOI: 10.3233/AIC-1994-7104.

[19] B. Efron and C. Morris. 'Data Analysis Using Stein's Estimator and Its Generalizations'. In: *Journal of the American Statistical Association* 70.350 (1975), pp. 311–319. DOI: 10.1080/01621459.1975.10479864.

[20] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau and C. Gagné. 'DEAP: Evolutionary Algorithms Made Easy'. In: *Journal of Machine Learning Research* 13 (2012), pp. 2171–2175.

[21] L. Perron and F. Didier. *CP-SAT*. Google OR-Tools. URL: https://developers.google.com/optimization/cp/.

[22] A. Hogan et al. 'Knowledge Graphs'. In: *ACM Computing Surveys* 54.4 (2021), 71:1–71:37. DOI: 10.1145/3447772.

[23] N. Reimers and I. Gurevych. 'Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks'. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410.

[24] T. Saito and M. Rehmsmeier. 'The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets'. In: *PLoS ONE* 10.3 (2015), e0118432. DOI: 10.1371/journal.pone.0118432.

[25] G. W. Brier. 'Verification of Forecasts Expressed in Terms of Probability'. In: *Monthly Weather Review* 78.1 (1950), pp. 1–3. DOI: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

[26] J. E. Smith and R. L. Winkler. 'The Optimizer's Curse: Skepticism and Postdecision Surprise in Decision Analysis'. In: *Management Science* 52.3 (2006), pp. 311–322. DOI: 10.1287/mnsc.1050.0451.

[27] L. Zheng et al. 'Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena'. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023.

A Project proposal

The proposal as submitted on 11 September 2026; pages 1–3 of the original, the fourth being blank.

FILING REF:

PROJECT PROPOSAL

Date of proposal: 11/09/2026

Project Title: AnchorAI

Group ID (As Enrolled in Canvas Class Groups): Gate 0

Group Members (name , Student ID):

- ABDULAZIZ ALI S HASANAIN (**A0350856X**)
- ASIRI MOHAMMED YAHYA A (**A0351967N**)
- BINMANEEA KHALID WALEED M (**A0355712E**)

Sponsor/Client: *(Company Name, Address and Contact Name, Email, if any)*

Name: Aramco
Address: Dhahran 31311, Kingdom of Saudi Arabia
Contact Name: Alshahrani, Homoud T
Email: homoud.alshahrani@aramco.com

Background/Aims/Objectives:

Gym members who stop attending cancel weeks or months later. That lag is what we target. The Retention Intelligence System segments members by behavior, predicts who will leave, and decides who to contact given a fixed budget and limited staff hours. Retention is the industry's biggest cost. One in three members leaves yearly, half of new joiners within six months, taking both dues and ancillary spend with them.

Current practice fails three ways:

- Calendar campaigns contact everyone, wasting budget on members who would have stayed.
- Threshold rules ignore trend. Twelve visits a month dropping to four beats a steady two, but rules treat them alike.
- Vendor scores give a number, not a decision. Staff cannot interrogate the flag, so they ignore it.

Our goals:

- Segment members interpretably using unsupervised learning.
- Predict churn with a calibrated gradient boosting and logistic regression ensemble.
- Turn predictions into a weekly action plan where every recommendation carries a challengeable justification.

FILING REF:

Market Analysis:

The vendor landscape splits into two layers. The system of record, covering membership management, booking, payments and access control, is dominated by ABC Fitness including Glofox and Trainerize, alongside Mindbody, Xplor Technologies, Perfect Gym, Virtuagym and Daxko. Above it sits a thinner intelligence layer that bolts on rather than replacing the system of record. Keepme is the clearest example, best known for its predictive Keepme Score, targeting multi site operators and pricing by custom quote rather than published rates. Glofox offers a built in "At Risk" churn report. Current buyer guides for AI in gyms place Keepme at the head of the predictive retention and churn analytics category, with Replify leading AI reception and sales.

Four gaps define our opportunity:

- Competitors score risk but do not publicly expose constrained allocation, meaning the decision of which flagged members to contact and with which offer given a fixed budget and finite staff hours.
- Vendor messaging is uniformly built around targeting high risk members, despite academic evidence that risk ranking is suboptimal.
- Scores trained on tens of millions of profiles are powerful but unauditable, and mid market operators repeatedly cite staff distrust of black box flags.
- The intelligence layer is priced for larger chains. Keepme is generally recommended only once an operator reaches 5+ locations with real lead volume and a sales team to feed, leaving the one to five club operator, the bulk of the market by count, underserved.

We position this project as an explainable, budget aware retention planner for mid market chains, differentiating on transparent decision making rather than on model accuracy, where incumbents with far larger training corpora will always win.

Sources: *Health and Fitness Association 2025 Fitness Industry Benchmarking Report; Keepme and Glofox product documentation; industry buyer guides published in 2026.*
https://www.healthandfitness.org/health-fitness-association-releases-2025-fitness-industry-benchmarking-report/
https://www.keepme.ai/blog/making-the-most-of-your-data
https://www.glofox.com/blog/how-glofoxs-ai-can-mitigate-member-churn-and-boost-retention/
https://www.replify.ai/ai-sales-service-blog/best-ai-for-gyms-complete-buyers-guide

Project Description:

The system is scoped to cover four technique groups. Each group maps to a working component:

- **Decision automation:** A forward chaining rule engine encodes retention playbooks and commercial guardrails staff can read and edit. Fuzzy inference assigns engagement tiers, avoiding threshold cliffs. A case based reasoner picks the offer by matching similar past members and their outcomes.

- **Resource optimization:** Allocating a fixed budget across members, offers and channels is a constrained assignment problem, solved with a genetic

FILING REF:

algorithm. Fitness is expected retained value net of cost, penalized for budget, staff hour, contact fatigue and inventory violations. An exact solver benchmarks small instances so we can report the optimality gap.

- **Knowledge discovery:** K means and Gaussian mixture segmentation, a calibrated churn ensemble, an uplift model for treatment effects, and class recommendations matched to member history.

- **Cognitive techniques:** A knowledge graph of members, clubs, classes, instructors and offers propagates cohort signals such as a cancelled class slot or departed trainer. A natural language assistant answers staff questions, grounded on the rule trace and graph.

Data Collection and Preparation

We will use a hybrid strategy. The public Kaggle dataset of 4,000 members benchmarks our pipeline, but it is a single month snapshot with no intervention data, so trajectory clustering and uplift modeling are impossible on it alone.

We therefore will also build a synthetic longitudinal simulator where each member is an agent with latent habit strength, motivation and social embeddedness, visiting via a process that decays during gaps and strengthens with attendance. It is calibrated to published benchmarks: roughly 66 percent annual retention, a first 90 day cliff, elevated January churn. Treatment effects are deliberately heterogeneous so the uplift model has real structure to recover. All parameters go in the appendix. Rule base and knowledge graph content come from two to four interviews with club managers or trainers, plus published effect sizes.

Dataset: Gym Customers Features and Churn, Adrian Vinueza, CC BY-NC-SA 4.0 https://www.kaggle.com/datasets/adrianvinueza/gym-customers-features-and-churn

Features are built as of a scoring date across recency, frequency, regularity, class engagement, relationship, commercial and lifecycle families. Trend and regularity carry the argument: absolute activity is a weak signal, deviation from a member's own pattern is a strong one.
Two preparation issues:

- **Leakage.** We exclude features generated by the cancellation process itself, such as freeze requests and payment method removal. These are the label in disguise, and likely explain part of the inflated accuracy in existing gym churn papers.
- **Censoring.** Members still active at window end are treated as unobserved, not silently coded as non churners.

Identifiers are pseudonymized, no health status is inferred, fairness is checked across age and gender, and contact fatigue caps are hard constraints in the optimizer.

B Mapping to MR, RS and CGS

All four technique groups the module requires are covered: decision automation, resource optimisation, knowledge discovery and cognitive systems (MR Machine Reasoning, RS Reasoning Systems, CGS Cognitive Systems; D is the course day). Table 8 lists every change from the proposal and its reason.

Table 7 Techniques as built, by course and evidence.

Component	Technique as built	Course	Evidence
Data preparation	stratified split; SMOTE-NC inside folds compared with class weights; CRISP-DM	RS D2	E2
Features	trend and ratio features, scaling, leakage check	MR D4, RS D2	E2, leakage check
Segmentation	K-means, silhouette, stability (adjusted Rand index), profiles	MR D4	E1
Churn model	class-weighted logistic regression and LightGBM ensemble, Platt calibration; F1, PR-AUC, Brier	MR D2, D4	E2
Driver explanations	averaged logistic-regression terms and TreeSHAP contributions per member	MR D3, D4	E2 drivers
Scenario layer	synthetic gym and randomised history with known effects	MR D3	scenario layer
Uplift baseline	T-learner (self-researched)	MR D4	E3
Offer reasoner	case-based reasoning: retrieve, reuse, revise (shrinkage), retain	MR D2	E3
Plan optimiser	genetic algorithm with repair, elitism, spillover and diminishing returns	RS D4	E4, E5
Exact benchmark	CP-SAT constraint optimisation	MR D1, RS D1	E5
Policy evaluation	simulation-based comparison against three baselines	RS D1	E4
Rule engine	forward chaining over derive, playbook and guardrail rules; salience; traces; versions	MR D2, RS D1	E6
Knowledge graph	Neo4j and Cypher; multi-hop cohort signals; read-only parameterised queries	MR D3, RS D5	E7
Intent detection	sentence-embedding centroids with an unknown threshold; slot filling	CGS D1	E8
Grounded generation	deterministic tool, then an LLM phrases cited evidence only	CGS D1, D3	E8
Assistant evaluation	keyword baseline; static checks; calibrated LLM-as-judge; local and hosted models	CGS D2, D3	E8
Message drafts	LLM drafts with a rule check and a template fallback	CGS D1	drafts in the application
Overall	hybrid machine learning, rules, optimisation, knowledge graph and language interface with human review	RS D1, CGS D1	E4, E6, E7

Deviations from the proposal

Table 8 Changes from the proposal and the presentation deck.

Proposal or deck	Built	Reason
A neural network picks who to contact (deck)	genetic algorithm, with CP-SAT as exact benchmark	a neural network learns; it cannot enforce budget or staff limits
Agent-based longitudinal simulator	real Kaggle members plus a thin scenario layer	lower risk; segmentation and churn stay results on real data
SMOTE to a 50/50 balance	SMOTE-NC inside folds compared with class weights; class weights kept; Platt calibration	oversampling inflates the probabilities the planner multiplies
F1 and silhouette as the metrics	F1, PR-AUC, Brier, calibration and fairness gaps	the plan consumes probabilities
Fuzzy inference for engagement tiers	not built	not taught in the three courses
K-means and Gaussian mixture	K-means only	matches the deck and MR D4
CBR and uplift as separate components	CBR is the uplift estimator; the T-learner its benchmark	both estimate the same quantity
Recency and regularity features; censoring	not built; censoring dropped	the snapshot has no visit dates and one binary label
Exclude cancellation-process fields	a with-and-without check on same-month columns	the snapshot's leakage risk is timing
Class recommendations; surrogate decision trees; graph embeddings	not built	stretch items
Interviews with managers	simulated by an AI model, and disclosed	no real operator was reached
The LLM calls tools itself	intent router before the LLM; tool calling optional	small local models are unreliable at tool calling

C Installation and user guide

Install. Prerequisites: Git, uv 0.13+, Node.js 20+, Docker with Compose, and optionally an OpenAI-compatible LLM endpoint (the default is a local Ollama `qwen2.5:7b-instruct`). Download *Gym Customers Features and Churn* from Kaggle [8] into `SystemCode/data/raw/`; the data is CC BY-NC-SA 4.0 and not redistributed.

```
git clone https://github.com/sgtaziz/IRS-PM-2026-08-17-ISFT-GRP-16-AnchorAI.git AnchorAI
cd AnchorAI
cp .env.example .env                  # set ANCHORAI_JWT_SECRET and LLM_*
docker compose up -d neo4j            # knowledge graph
cd SystemCode/backend && uv sync
uv run anchorai setup                 # data, models, seeded DB, graph
uv run uvicorn anchorai.api.main:app  # API on :8000
cd ../frontend && npm install
npm run dev                           # web app on http://localhost:5173
```

Sign in as `admin`, `manager`, `staff1` or `staff2` with the password `anchorai`. The pipeline, API and web application run without Neo4j or a language model, with the graph panels and assistant shown as unavailable. `uv run pytest -q` runs the 195 offline tests; `uv run anchorai db reset` restores the demo database; the administrator's Settings page checks every service.

Docker for everything. Build the data on the host first, because the demo seed needs the tables; the API container migrates and seeds the database on start. Then load the graph from the host, or with *Reload knowledge graph* on the Events & roster page.

```
cd SystemCode/backend && uv sync && uv run anchorai data build
cd ../.. && docker compose up --build -d
cd SystemCode/backend && uv run anchorai kg load
```

Table 9 Demo users. The password of every demo user is `anchorai`, or the value of `ANCHORAI_DEMO_PASSWORD`.

User	Role	Lands on, and can use
`admin`	Admin	Dashboard; everything, including Rules, Offers & constraints, Data & models, Users, Audit log, Settings
`manager`	Manager	Dashboard; Segments, Members, Weekly plan, Scenarios, Knowledge graph, Events & roster
`staff1`, `staff2`	Staff	My contacts; Member 360, Assistant

Checking the install. In the application, the administrator's Settings page shows whether the language model, Neo4j, the models and the database are working, and tests the LLM connection.

Using the application. AnchorAI has three roles, and the role decides which pages appear in the sidebar; the server checks the role again on every request. A session lasts 12 hours. Every page has a member search at the top (Ctrl+K), a light, dark or system theme, and, except on the Assistant page itself, a round button that opens the assistant beside the current page.

Manager: check the risk. The *Dashboard* (Figure 10) shows members at high risk (a churn probability of 0.6 or more), the monthly revenue at risk (the sum of churn probability × monthly fee), the share of high-risk members covered by the latest approved plan, the latest week's contacted, reached and

Table 10 Troubleshooting.

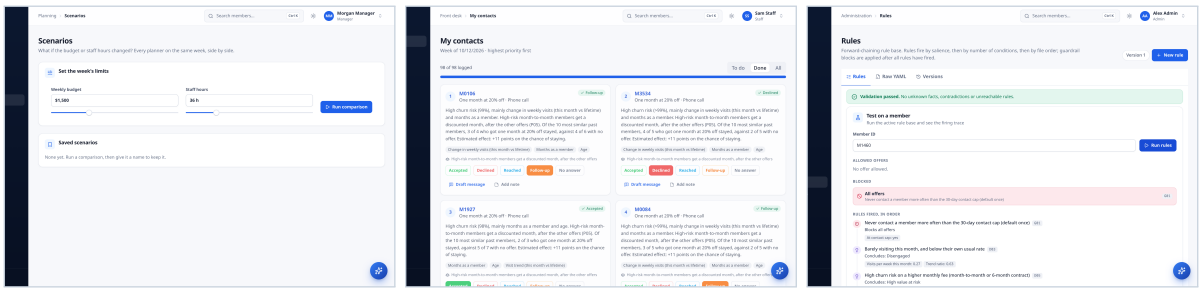
Symptom	Fix
'Expected exactly one Kaggle CSV'	put exactly one file, `gym_churn_us.csv`, in `SystemCode/data/raw/`
Pages say the data tables are not built (HTTP 503)	run `uv run anchorai data build`
Demo users cannot sign in	run `uv run anchorai db reset`
Graph panels or assistant show 'unavailable'	start Neo4j or the LLM endpoint; the rest keeps working
`npm` fails with an engine or native-binding error	upgrade to Node 20 or newer, delete `node_modules` and reinstall
Docker reports a missing host–sandbox network interface	the kernel was updated without a reboot; reboot
Port 8000 or 5173 is in use	stop the other process, or change the port

retained counts, the risk distribution, the segment mix and outcomes by week. *Segments* shows one card per segment with its size, churn rate, members at high risk now, revenue and a profile against all members. *Members* filters by ID, segment, risk band, club and tenure, sorts by any column and exports every matching member to CSV. A row opens *Member 360*.

Manager: build, review and approve the plan. On *Weekly plan* (Figure 10), enter the weekly budget and staff hours (they start from the latest plan, or $1,500 and 36 hours), choose a planner (the genetic algorithm by default, which considers members who train together; exact; or greedy, the quickest) and select *Generate plan*. Four tiles sum it up: members to contact, spend, staff time and extra members expected to stay, each compared with last week. *Why* on a row opens the member's summary, churn drivers, the rules that fired, any gym events and the ten most similar past members with what happened to them. While the plan is a draft you can remove a member, add a contact or swap an offer; each change needs a reason and is checked against the rules at once (red: blocked, with the rule; amber: allowed but not proposed by a playbook; green: allowed and proposed). *Re-optimise the rest* keeps your changes and re-plans what is left of the budget, staff time and stock. *Approve plan* locks it; only one plan per week can be approved. Contacts are then shared out, most valuable first, to the staff member with the most free time who has the right skill. Drag a contact to another staff member, re-run the assignment, and print or download each person's call sheet.

Manager: what-ifs and events. *Scenarios* (Figure 19a) runs five planners (contact everyone, riskiest first, greedy, exact and the genetic algorithm) on the same week at a budget and staff hours you choose, compares contacts, spend, staff time, extra members, members per $1,000 and per staff hour, and saves named comparisons. Nothing here changes the plan. On *Events & roster*, record what really happened: add instructors and classes, change who teaches a class, mark an instructor as left or a class as cancelled for a given week, or reinstate them. The graph is rebuilt after each change, and rules that read graph signals then fire for the affected members; churn scores do not change. On *Knowledge graph* (Figure 9a), simulate an instructor leaving or a class closing to see who would be affected, how many are high risk and which rules would newly fire, without changing anything.

Staff: contacts, drafts and the assistant. *My contacts* (Figure 19b) lists the members assigned to you in this week's approved plan, highest priority first, each with the offer and channel, why the member is on the list, talking points from their risk drivers and the rule that proposed the offer. Log an outcome in two taps: *Accepted*, *Declined*, *Reached* (no decision yet), *Follow-up* or *No answer*, optionally with a note. Accepted and declined outcomes become new past cases, and every logged contact counts towards the contact limits, so the member is not contacted again too soon. *Draft message*, next to an allowed offer in a member's rule trace, writes a short SMS, email, call script or in-person note that you edit and



(a) Scenarios: set a week's budget and staff hours and compare five planners side by side. **(b)** My contacts: the staff call list with outcomes logged. **(c)** Rules: testing the rule base on one member shows the same trace as Member 360.

Figure 19 One page for each role: manager, staff and administrator.

copy into your own tool; AnchorAI never sends anything, and a draft never mentions churn, risk or scores. In the *Assistant* (Figure 9b), type a question and press Enter. It answers about member risk, what to offer, comparisons, ranked lists, who an event affects and, for managers, the plan and what-if budgets or staff hours, always from AnchorAI's own data. Numbers in an answer are chips that show the evidence behind each sentence. When nothing supports an answer it says 'I don't know based on the data I have'. It cannot change anything and does not remember a conversation after a refresh.

Administrator. The *Rules* page (Figure 19c) lists every rule with its kind, salience, conditions, action and source, under a validation banner. Toggle a rule on or off, add or edit one in a form that validates as you type, or edit the YAML directly; every save needs a note and creates a new version, and nothing is overwritten. *Test on a member* runs the saved rules, or an unsaved draft, on any member and shows the same trace as Member 360. *Versions* compares any two versions line by line and rolls back to an older one. *Offers & constraints* edits the offer catalogue (cost, staff time, stock, channels, required skill, eligibility, active), each staff member's weekly hours and skills, and the planning defaults: weekly budget, staff minutes and the contact caps the guardrails read. *Data & models* replaces the source file with a validated new Kaggle file, rebuilds data and models with any seed in the background (42 reproduces this report), reloads the graph, and shows the model card. *Users* creates accounts, changes roles and deactivates users; *Audit log* records every write with who, when, and the values before and after; *Settings* is a read-only health check of the language model, the knowledge graph, the models and data, and the database.

D Use of AI tools

The team used an AI coding assistant (Claude Code) throughout and logged every meaningful use in the repository, with what it was used for and how the result was checked; the log is reproduced below. Three principles applied. Every output was reviewed by the member responsible, and every number in this report was re-measured against the committed results and notebooks rather than taken from the tool. Design decisions, such as the choice of estimator, planner defaults and experiment axes, were made by the team. And where AI stood in for people, it is disclosed: the manager interviews that shaped the rule base were simulated by an AI model and are treated as plausible opinion, not evidence (Section 4.2); 62 of the assistant's 86 test questions were AI-written and are reported separately from the 24 written by team members (E8). The report text was drafted with AI assistance from the team's own section drafts and reviewed by the team. No AI-generated speech is used in the videos. In the log, Abdulaziz is Abdulaziz Ali S Hasanain, Asiri is Asiri Mohammed Yahya A, and Mokdy is Binmaneea Khalid Waleed M.

Table 11 The AI-use log, generated from `docs/AI_USE.md`; the tool in every row is Claude Code.

Date	Member	Used for	What we changed or verified
10-13	Abdulaziz	Course review, project plan, repo scaffold	Reviewed plan with team; adjusted scope
10-15	Abdulaziz	SQLite app DB, Alembic migrations, seeds, auth	Reviewed and tested
10-16	Mokdy	Member intelligence data-table contract, fixture and scenario v0 build	Reviewed; verify on real Kaggle CSV
10-16	Mokdy	Member intelligence A2–A8: features, segmentation, churn model, scenario v1, T-learner, notebooks E1/E2, member/admin APIs, web shell and pages, docs	Reviewed results against notebooks; re-measured reported numbers
10-16	Abdulaziz	Rule engine seeds, knowledge graph setup	Tested; thresholds pending
10-16	Abdulaziz	WebUI redesign	Reviewed screenshots; reviewed for regressions
10-16	Abdulaziz	Assistant chat	Reviewed and tested
10-16	Abdulaziz	Simulated manager interviews	Chose thresholds from interview and data checks; reviewed
10-17	Mokdy	Member intelligence handoffs #1–#5, #16, #18, #20: gym-event ground truth (`true_churn`), out-of-fold churn scores, signed `visit_change`, zero-visit attendance fix, phone check-ins; cross-stream review of C1–C5 (three AI review agents, each finding re-run or traced before filing); drafts of the Member intelligence report sections and user-guide pages	Re-measured every reported number against `notebooks/results/` and the model card (table cross-check script); mutation-tested the new regression tests; ran each commit's tests in isolation; browser check in both themes for all three roles; fresh-clone install test. Report and user-guide text is an AI draft for Mokdy to review and edit before submission
10-17	Abdulaziz	Review finding fixes	Reviewed and tested
10-17	Abdulaziz	Graph explorer class/instructor search + E8 prompt variant	Reviewed, queries checked on live graph
10-19	Asiri	Decision and optimisation handoff review: #6, #12, #13, #17, #26 replies and the #30 finding (override check misses no-phone members)	Each reply checked against the running API and the rule engine; candidate-set decision mine
10-19	Asiri	B1/B5 plan model notebook: options, objective, feasibility check, policy evaluator, baselines, first E4 grid; moved into `anchorai.optimisation` with feasibility and evaluator tests	Ran every cell and checked the outputs; tests checked by removing each limit check in turn; decisions (H = 6, baseline offer, E4 axes) mine
10-20	Asiri	Decision and optimisation walking skeleton: POST /api/plan saving plans to SQLite, and a bare Plan builder page	Checked end to end in the browser; tests for roles, feasibility and audit
10-20	Asiri	CBR uplift reasoner (B4) and E3: b_cbr notebook, three pre-registered variants, null test, plan quality vs T-learner	Ran the notebook; caught a flawed first version with the null test; decided to keep CBR per PLAN and flag the T-learner question to the team (HANDOFFS #33)
10-20	Asiri	Genetic algorithm planner (B2) with spillover and diminishing returns, and E4 on CBR, T-learner and perfect-knowledge estimates	Ran the notebook; chose the spillover design (#7); reported the optimiser's-curse finding honestly
10-20	Asiri	CP-SAT exact planner (B3), random-instance generator (B1) and E5: optimality gap, speed, seeds, sensitivity, ablations	Ran the notebook; package results checked identical to the notebook's

Date	Member	Used for	What we changed or verified
10-20	Asiri	Decision and optimisation backend (B6): explanations, overrides with live rule checks, approval, assignment and call sheets, my contacts, outcome logging with CBR retain, background jobs, re-optimise, saved scenarios, health checks	Tests for each rule, several checked by breaking the rule on purpose; reviewed each change
10-20	Asiri	Decision and optimisation pages (W14–W17, W20, W21, W45) and the demo seed layer (#8)	Tested every page in the browser by role, at tablet and phone sizes and in both themes; checked A's dashboard and C's assistant read the demo week
10-20	Abdulaziz	E6–E8 reruns on B's planner	Reviewed, tested, fixed what-if parsing bug found in E8
10-22	Asiri	Report sections: objectives and success measures, system architecture with two Graphviz figures, decision and optimisation design, experiments E3–E5, and the planner pages of the user guide	Directed structure and scope; reviewed each section and asked for changes; numbers checked against the results files and claims against the code; chose to cite only techniques not taught in class; references verified online
10-22	Mokdy	Member intelligence report: rewrote and extended the data preparation, knowledge discovery and E1–E2 sections; drafted the implementation section, the scenario-layer appendix and Member intelligence's input to the conclusions; corrected the installation guide (Node 20, knowledge graph under Docker); Member 360 renders Decision and optimisation's similar-cases and contact-history panels instead of raw JSON (HANDOFFS #35)	Every number checked against `notebooks/results/`, the notebooks' saved outputs and the figures; claims traced to the code; `npm run build` passes. Report text is an AI draft for Mokdy to review and edit before submission
10-23	Mokdy	Tagging the intents of the 15 E8 held-out questions (HANDOFFS #14) and adding them to `e8_questions.yaml`	Questions written by Mokdy; Claude only assigned intents and notes, wording kept as typed
10-23	Mokdy	LaTeX group report (`ProjectReport/latex/`): NUS-styled template, all chapters condensed from the three streams' section drafts into one voice, pgfplots charts read from `notebooks/results/`, vector figures regenerated by `scripts/figures.py`, TikZ diagrams, app screenshots captured from a local run, appendices; E8 part 1 rerun with the 15 new held-out questions	Fresh data build reproduced the model card exactly (threshold 0.404 confirmed); every decimal in the report cross-checked against results files, notebook outputs and the model card; all DOIs resolved; 195 backend tests passed. Report text is an AI draft from the team's sections, for the team to review

E Individual project reports

Abdulaziz Ali S Hasanain

Individual reflection

My contribution. I helped set up the project plan, split the work into three streams with shared contracts, and scaffolded the repository. I then led the knowledge and interaction part of the system. I built the rule engine, which uses forward chaining, along with its validator, guardrails and rule versioning. I built the Neo4j knowledge graph and the cohort signals it feeds back into the rules. I also built the staff assistant, which routes a question to a tool and answers only from that tool's output. Finally, I built the app database with migrations, seeds and login by role, ran experiments E6 to E8, and did general UI/UX work in the web app.

What I learned. The most useful lesson was how the three course modules fit together in one system. From Machine Reasoning, writing rules as data with a validator taught me that a rule base needs the same care as code. It needs tests that state which rules must fire, checks for contradictions, and versions that can be rolled back. From Cognitive Systems, the knowledge graph showed me that structure can add reach rather than accuracy. It flagged 243 members at low risk whose routine had just been disrupted, members the churn model would never rank. The assistant taught me that grounding matters more than the model. Its answers were 95.1% supported by evidence with both a hosted and a local model, but its intent routing was no better than keywords on unseen questions, so the weak point was the routing and not the language model.

The hardest part was the trade off between guardrails and plan value. Guardrails removed 31.3% of proposals, almost all of them repeat contacts within 30 days. When staff time was tight this cost almost nothing, but at 120 staff hours the plans without guardrails kept 6 to 14 more members, because our scenario has no contact fatigue effect to penalise repeat contacts. An evaluation can only measure what it models, so we claim safety and auditability for the guardrails and not a retention gain. Next time I would build the cost of the behaviour a rule protects against into the evaluation from the start.

Integration was the other big lesson. Machine learning, rules, case based reasoning and optimisation each worked alone well before they worked together. Agreeing on schemas early, and keeping the system running when the LLM or the graph was unavailable, saved us more time than any single model did.

Applying it. I work as a product development engineer at LAB7, the venture building arm of Aramco, where many of the startups we build and support are developing AI systems. From this project I take a checklist for them. Keep business rules explicit and versioned beside the model, ground any LLM feature in tool output and measure its faithfulness, use a knowledge graph where relationships carry signal, and decide early how a system will be evaluated when real outcomes are not yet available. Above all, I now push for explainability as a design requirement, since staff will only act on a plan they can question.

Asiri Mohammed Yahya A

Individual reflection

My contribution. I led the decision and optimisation part of the system, which turns risk scores and rules into a weekly contact plan. I built the case-based reasoner that estimates how much each offer would help each member, using 2,400 past cases, and shows the most similar past members as evidence for each recommendation. I built the genetic algorithm planner with its repair operator and greedy seeding, an exact CP-SAT planner that also serves as its benchmark, the baseline policies, and a policy evaluator that grades any plan against the scenario's ground truth. I ran experiments E3 to E5. On the app side I built the plan workflow: the plan builder, overrides checked live against the rules, approval, automatic assignment to staff with call sheets, scenario comparison, the staff contact list with outcome logging that feeds back into the case base, and the settings and health page. I also wrote the architecture, objectives, optimisation and E3–E5 parts of this report.

What I learned. I came to this project from software engineering, new to AI, and that was challenging but exciting. My engineering habits helped: I prepared tests first and never reported results I had not tested. Several of the tests were checked by breaking the rule they guard on purpose, to prove they could fail.

The biggest surprise was the null test. My first version of the case-based reasoner scored well against the true effects. But when I shuffled which members received which offer, so that there was nothing left to find, it still "found" effects: it was measuring churn risk, not the offer. That taught me not to trust a good result until I am sure what it is measuring and that it behaves as expected when there is nothing to find.

Building CBR also changed how I understood it. I had thought it was the same as kNN. kNN only looks up the most similar cases. CBR works from a case base of previous cases, both successes and failures, and goes through a cycle: it reuses them, revises the estimate, and retains each new outcome for future use.

The experiments taught me that a better optimiser cannot make up for weak estimates. With perfect knowledge of the effects, the genetic algorithm kept as many members as a simple greedy planner or more. On the case-based estimates it kept fewer (16.7 against 21.0 at the default week), because a stronger search also finds the errors in its inputs. Retaining new cases has a trap too. The planner chooses whom to contact, so its own outcomes are not a fair sample, and only a random control group would let the system keep learning without bias.

Applying it. I already work in product development supporting startups, and with the current boom in AI many of them are building AI systems. This project lets me understand how such systems work underneath, and how I can contribute beyond the engineering. I will ask what a result is actually measuring, test that a method finds nothing when there is nothing to find, and check the quality of the estimates before trusting an optimiser built on them.

Binmaneea Khalid Waleed M

Individual reflection

My contribution. I led the member intelligence part of the system. I defined the data-table contract that the other parts build against, and I built the data layer. It joins the Kaggle members with a synthetic scenario of clubs, classes, instructors, offers, past retention contacts and gym events, and it carries ground truth so that uplift and event effects can be measured. I built the feature engineering with its leakage check, the K-means segmentation (E1) and the calibrated churn model (E2). The model averages logistic regression and LightGBM, scores training members out of fold, and explains each score with per-member drivers. I also built the T-learner uplift baseline. On the app side I built the member and admin APIs, the data upload and rebuild flow, the dashboard and the member pages, including Member 360, which brings all three parts together. Finally, I reviewed the decision automation work, wrote the installation guide, and wrote the data, member intelligence and E1–E2 parts of this report.

What I learned. The most useful lesson was that a good score is not the same as a good model. The churn model reached an F1 of 0.908 against 0.549 for a simple visit rule, but much of that came from a trend signal that compares this month's visits with the member's average. Without that signal and its close relatives, F1 fell to 0.817, which is the figure I would quote to a gym that measures visits differently. Calibration mattered as much as ranking. The planner multiplies probabilities by uplift and cost, so a well-ranked but badly calibrated score would have produced plans that look optimal and are not.

The scenario layer was the hardest part. The Kaggle data has no offers, no past contacts and no gym events, so I had to generate them, and everything generated is an assumption. I kept the ground truth separate from the training data, and I recorded which effects were invented and how strong they were. Even so, the uplift baseline recovered only part of the true ranking (Spearman 0.39 overall, 0.61 for free personal training), and the small contact history could not confirm whether it or the case-based reasoner gave better plans. Uplift is much harder to learn than risk, and an honest evaluation has to show that.

The fairness check was a lesson too. Recall for members aged 31 and over was 0.52 because they rarely leave, so the same model treats age groups differently without any change to its inputs. I now check groups separately as a matter of course.

Applying it. From this project I take a habit of checking for leakage before trusting a strong result, calibrating any score that feeds a decision, and stating plainly what a synthetic evaluation can and cannot show. Agreeing on the data-table contract early also let the three parts work in parallel, and I would do that first on any team project.